ЗВІТ

про виконання лабораторної роботи №10

«Локальне збереження даних у Flutter-застосунку»

з дисципліни

«Програмування мобільних додатків»

Студента групи КН-2327Б

Багрія-Белза Андрія

Завдання: 1. Додати у проєкт пакет для локального збереження даних (SharedPreferences).

2. Реалізувати збереження даних користувача з екрана реєстрації.

3. Зберігати такі поля: a) ім’я користувача; b) email; c) пароль або інше поле, передбачене формою реєстрації.

4. Реалізувати завантаження даних користувача після повторного запуску застосунку.

5. Відобразити збережені дані на екрані ProfileScreen.

6. Реалізувати додавання елементів у список обраного.

7. Зберігати локально лише список favorites ids.

8. Після запуску застосунку автоматично відновлювати список обраних елементів за збереженими id.

9. На екрані зі списком обраних відображати лише ті елементи, id яких містяться у списку збережених.

10. Перевірити, що після перезапуску застосунку дані профілю не втрачаються, а обрані елементи залишаються в списку favorites.

1 Написання коду-розв’язку завдання:

```
// === pubspec.yaml (фрагмент, п. 1 ТЗ) ===
dependencies:
  flutter:
    sdk: flutter
  shared_preferences: ^2.5.6  // локальне збереження даних

// === lib/models/user_data.dart ===
/// Модель даних користувача, які зберігаються локально у SharedPreferences
/// (п. 3 ТЗ ЛР 10: ім'я, email, пароль)
class UserData {
  /// Ім'я користувача
  final String name;

  /// Email користувача
  final String email;

  /// Пароль (поле, передбачене формою реєстрації)
  final String password;

  const UserData({
    required this.name,
    required this.email,
    required this.password,
  });

  @override
  String toString() => 'UserData(name: $name, email: $email)';
}

// === lib/services/user_storage_service.dart (п. 1–4, 6–8 ТЗ) ===
import 'package:shared_preferences/shared_preferences.dart';
```

```dart
import '../models/user_data.dart';

/// Сервіс локального збереження даних через SharedPreferences (п. 1 ТЗ ЛР 10):
/// - дані користувача з екрана реєстрації (ім'я, email, пароль) — п. 2, 3 ТЗ;
/// - список ідентифікаторів обраних елементів (favorites ids) — п. 6, 7 ТЗ.
class UserStorageService {
  // Ключі збереження даних користувача (п. 3 ТЗ)
  static const String keyName = 'user_name';
  static const String keyEmail = 'user_email';
  static const String keyPassword = 'user_password';
  static const String keyIsRegistered = 'is_registered';

  // Ключ списку favorites ids (п. 7 ТЗ)
  static const String keyFavoriteIds = 'favorite_ids';

  /// Збереження даних користувача, введених на екрані реєстрації (п. 2, 3 ТЗ)
  Future<void> saveUser({
    required String name,
    required String email,
    required String password,
  }) async {
    final prefs = await SharedPreferences.getInstance();
    await prefs.setString(keyName, name);
    await prefs.setString(keyEmail, email);
    await prefs.setString(keyPassword, password);
    await prefs.setBool(keyIsRegistered, true);
  }

  /// Завантаження даних користувача після повторного запуску (п. 4 ТЗ).
  /// Повертає null, якщо користувач ще не реєструвався.
  Future<UserData?> loadUser() async {
    final prefs = await SharedPreferences.getInstance();
    if (prefs.getBool(keyIsRegistered) ?? false) {
      return UserData(
        name: prefs.getString(keyName) ?? '',
        email: prefs.getString(keyEmail) ?? '',
        password: prefs.getString(keyPassword) ?? '',
      );
    }
    return null;
  }

  /// Прапорець наявності збереженого користувача (п. 4 ТЗ)
  Future<bool> isRegistered() async {
    final prefs = await SharedPreferences.getInstance();
    return prefs.getBool(keyIsRegistered) ?? false;
  }

  /// Отримання збережених favorites ids (п. 8 ТЗ)
  Future<List<String>> loadFavoriteIds() async {
    final prefs = await SharedPreferences.getInstance();
    return prefs.getStringList(keyFavoriteIds) ?? const [];
  }

  /// Збереження списку favorites ids (п. 6, 7 ТЗ)
  Future<void> saveFavoriteIds(List<String> favoriteIds) async {
    final prefs = await SharedPreferences.getInstance();
    await prefs.setStringList(keyFavoriteIds, favoriteIds);
  }

  /// Видалення даних користувача (вихід з облікового запису)
  Future<void> clearUser() async {
    final prefs = await SharedPreferences.getInstance();
    await prefs.remove(keyName);
    await prefs.remove(keyEmail);
    await prefs.remove(keyPassword);
    await prefs.remove(keyIsRegistered);
  }

  /// Видалення збережених favorites ids
  Future<void> clearFavorites() async {
    final prefs = await SharedPreferences.getInstance();
    await prefs.remove(keyFavoriteIds);
  }

  /// Повне очищення локального сховища (вихід + скидання обраного)
  Future<void> clearAll() async {
    final prefs = await SharedPreferences.getInstance();
    await prefs.clear();
```

```
  }
}

// === lib/screens/welcome_screen.dart (фрагмент, п. 4 ТЗ) ===
class _WelcomeScreenState extends State<WelcomeScreen> {
  final UserStorageService _userStorage = UserStorageService();

  @override
  void initState() {
    super.initState();
    _restoreSession();
  }

  // Якщо дані користувача вже збережені — сесія відновлюється
  // і застосунок одразу переходить у менеджер завдань (п. 4 ТЗ ЛР 10)
  Future<void> _restoreSession() async {
    final user = await _userStorage.loadUser();
    if (user != null && mounted) {
      Navigator.of(context).pushReplacement(
        MaterialPageRoute(
          builder: (context) => TaskManagerScreen(
            username: user.name,
            email: user.email,
            password: user.password,
            onToggleTheme: widget.onToggleTheme,
          ),
        ),
      );
    }
  }
}

// === lib/screens/register_screen.dart (фрагмент, п. 2–3 ТЗ) ===
  // П. 2, 3 ТЗ ЛР 10: дані користувача зберігаються у SharedPreferences
  Future<void> _submitForm() async {
    if (_formKey.currentState!.validate()) {
      final String username = _usernameController.text.trim();
      final String email = _emailController.text.trim();
      final String password = _passwordController.text;

      // Локальне збереження даних користувача (ім'я, email, пароль)
      await UserStorageService().saveUser(
        name: username,
        email: email,
        password: password,
      );
      if (!mounted) return;

      // Навігація на TaskManagerScreen із передачею даних користувача
      Navigator.pushReplacement(
        context,
        MaterialPageRoute(
          builder: (context) => TaskManagerScreen(
            username: username,
            email: email,
            password: password,
            onToggleTheme: widget.onToggleTheme,
          ),
        ),
      );
    } else {
      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(content: Text('Будь ласка, заповніть усі поля форми!')),
      );
    }
  }

// === lib/screens/login_screen.dart (фрагмент, п. 4 ТЗ) ===
  // П. 4 ТЗ ЛР 10: якщо користувач збережено у SharedPreferences, дані
  // входу звіряються зі збереженими та сесія відновлюється.
  Future<void> _submitLogin() async {
    if (_formKey.currentState!.validate()) {
      final String email = _emailController.text.trim();
      final String password = _passwordController.text;

      // Завантаження збереженого користувача з локального сховища
      final UserData? saved = await UserStorageService().loadUser();
      if (!mounted) return;
```

```
      // Ім'я користувача: зі збережених даних або з локальної частини email
      String username;
      if (saved != null) {
        if (saved.email != email || saved.password != password) {
          _showError('Невірний email або пароль');
          return;
        }
        username = saved.name;
      } else {
        username = email.split('@').first;
      }

      // Навігація на TaskManagerScreen із відновленими даними користувача
      Navigator.pushReplacement(
        context,
        MaterialPageRoute(
          builder: (context) => TaskManagerScreen(
            username: username,
            email: email,
            password: password,
            onToggleTheme: widget.onToggleTheme,
          ),
        ),
      );
    }
  }

// === lib/screens/task_manager_screen.dart (фрагменти, п. 6–9 ТЗ) ===
  // 1. Зчитування даних із JSON-файлу та відновлення favorites ids
  //    із SharedPreferences (п. 8, 9 ТЗ ЛР 10)
  Future<void> _loadTasksFromJson() async {
    final loadedTasks = await _storageService.loadTasks();
    // Відновлення обраних елементів за збереженими id
    final savedIds = await _userStorage.loadFavoriteIds();
    for (final task in loadedTasks) {
      task.isFavorite = savedIds.contains(task.id.toString());
    }
    setState(() {
      _tasks = loadedTasks;
      _favoriteIds = savedIds;
      _isLoading = false;
    });
  }

  // 3. Синхронізація favorites ids у SharedPreferences (п. 6, 7 ТЗ ЛР 10)
  //    – зберігається лише список ідентифікаторів обраних елементів
  Future<void> _persistFavoriteIds() async {
    final ids =
        _tasks.where((t) => t.isFavorite).map((t) => t.id.toString()).toList();
    _favoriteIds = ids;
    await _userStorage.saveFavoriteIds(ids);
  }

  // 4. Додавання/видалення елемента зі списку обраного (п. 6 ТЗ)
  void _toggleFavorite(TaskModel task) {
    setState(() {
      task.isFavorite = !task.isFavorite;
    });
    _persistTasks();
    _persistFavoriteIds();
    _showSnackBar(
      task.isFavorite
          ? 'Додано до обраного ❤️ (id збережено в SharedPreferences)'
          : 'Видалено з обраного 🤍 (id видалено з SharedPreferences)',
    );
  }

  // Фільтр «Обрані» (п. 9 ТЗ ЛР 10): лише ті елементи,
  // id яких містяться у списку збережених
  List<TaskModel> get _filteredTasks {
    switch (_currentFilter) {
      case TaskFilter.favorites:
        return _tasks
            .where((t) => _favoriteIds.contains(t.id.toString()))
            .toList();
      case TaskFilter.all:
      default:
        return _tasks;
    }
```

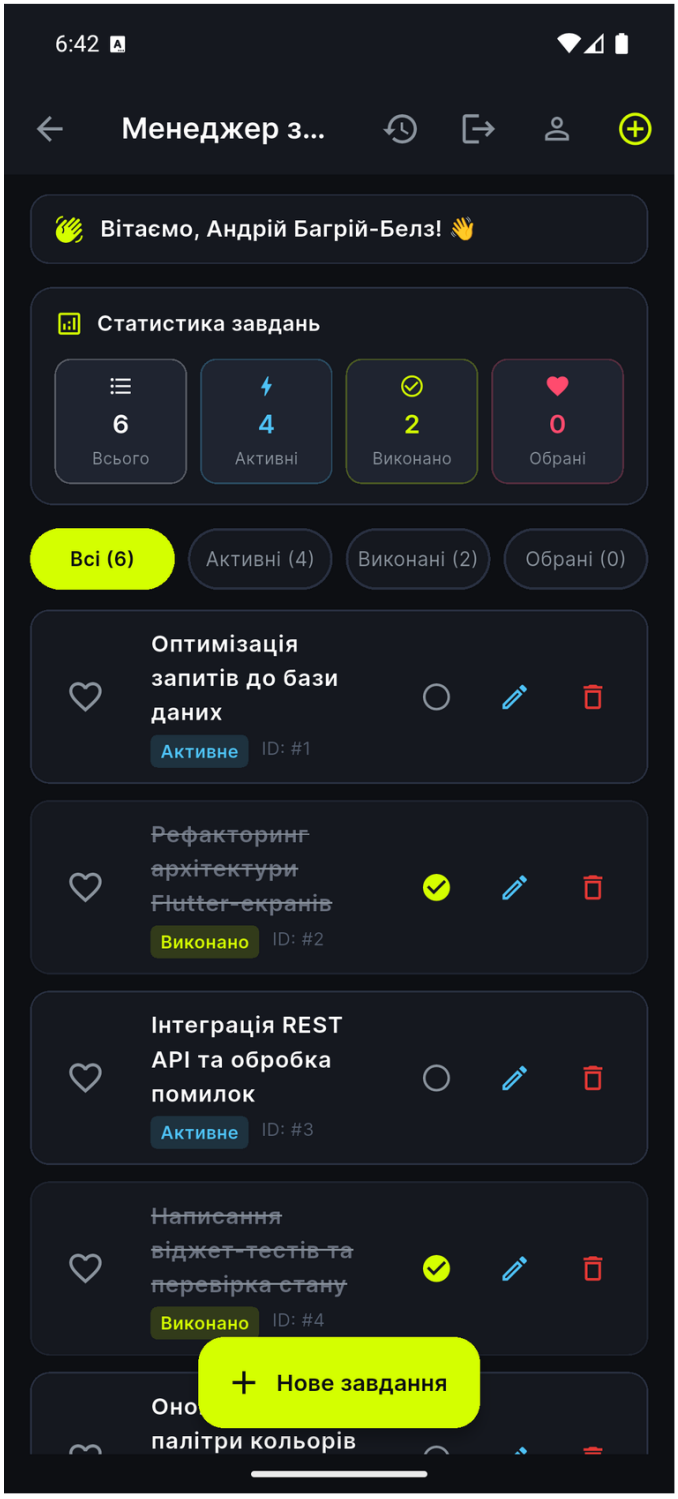
```
  }

// === lib/screens/profile_screen.dart (фрагмент, п. 5 ТЗ) ===
  /// Картка з даними користувача, збереженими у SharedPreferences (п. 4, 5 ТЗ ЛР 10)
  Widget _buildSavedDataCard(BuildContext context, bool isDark) {
    final bool hasSaved = _savedUser != null;
    return Material(
      color: AppColors.card(isDark),
      shape: RoundedRectangleBorder(
        borderRadius: BorderRadius.circular(14),
        side: BorderSide(color: AppColors.divider(isDark)),
      ),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.stretch,
        children: [
          // Заголовок картки
          Padding(
            padding: const EdgeInsets.fromLTRB(16, 14, 16, 4),
            child: Row(
              children: [
                Icon(
                  Icons.save_alt_outlined,
                  size: 20,
                  color: AppColors.secondary,
                ),
                const SizedBox(width: 8),
                Expanded(
                  child: Text(
                    'Дані зі сховища (SharedPreferences)',
                    style: TextStyle(
                      fontWeight: FontWeight.w600,
                      fontSize: 14,
                      color: isDark ? AppColors.textDark : AppColors.textLight,
                    ),
                  ),
                ),
              ],
            ),
          ),
          if (!hasSaved)
            const Padding(
              padding: EdgeInsets.fromLTRB(16, 4, 16, 14),
              child: Text('Збережених даних немає'),
            )
          else ...[
            _buildSavedRow(context, isDark,
                icon: Icons.person_outline, label: 'Ім'я', value: _displayName),
            _buildSavedRow(context, isDark,
                icon: Icons.alternate_email, label: 'Email', value: _displayEmail),
            // Пароль приховано крапочками, показ/сховати — за іконку
            Padding(
              padding: const EdgeInsets.symmetric(horizontal: 16),
              child: Row(
                children: [
                  Icon(
                    Icons.lock_outline,
                    size: 18,
                    color: AppColors.textSecondary(isDark),
                  ),
                  const SizedBox(width: 10),
                  Expanded(
                    child: Text(
                      _showPassword
                          ? _displayPassword
                          : '•' * _displayPassword.length,
                      style: const TextStyle(fontWeight: FontWeight.w600),
                    ),
                  ),
                ],
              ),
            ),
          ],
        ],
      ),
    );
  }

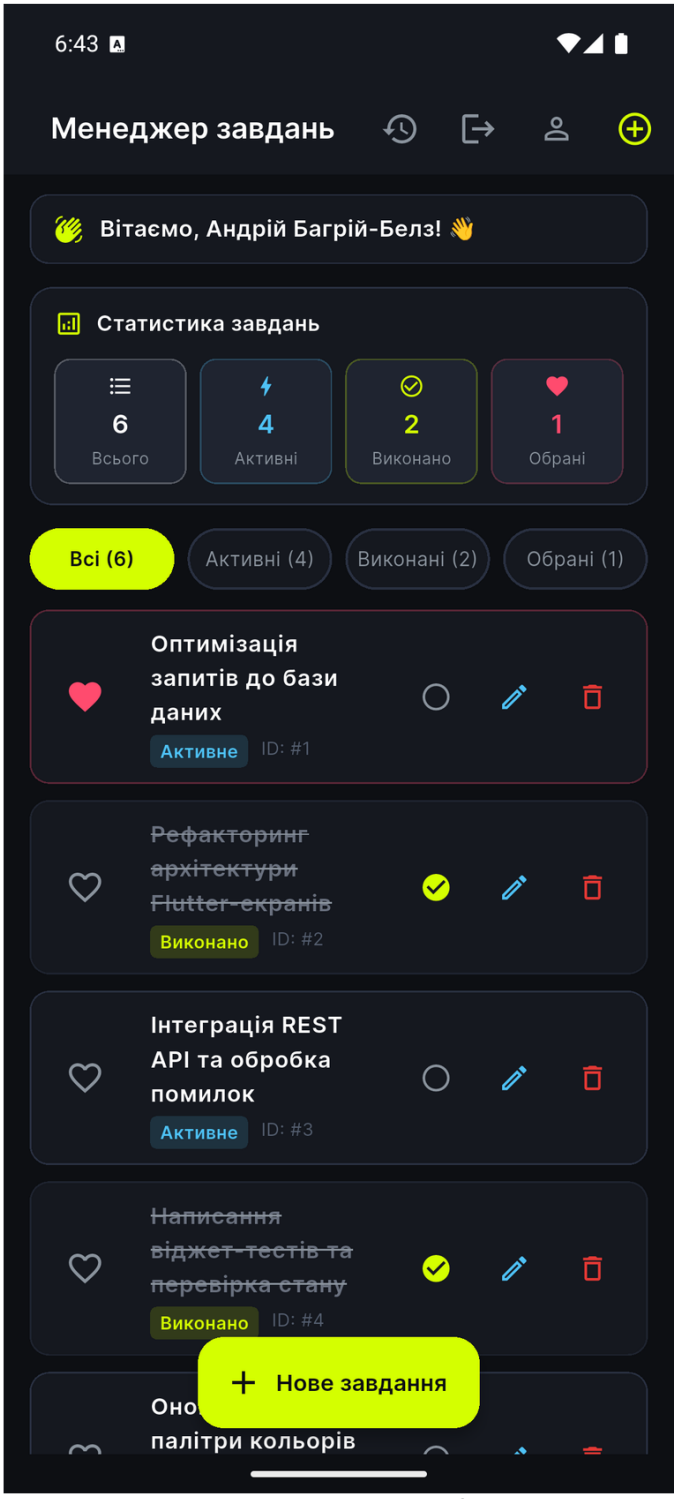
  // Вихід: очищення локального сховища та повернення на вступний екран
  void _confirmLogout(BuildContext context, bool isDark) {
```

```
    // діалог підтвердження; за кнопкою «Вийти»:
    //   final navigator = Navigator.of(context);
    //   navigator.pop();
    //   await _userStorage.clearAll();
    //   if (!mounted) return;
    //   navigator.pushAndRemoveUntil(
    //     MaterialPageRoute(
    //       builder: (context) =>
    //           WelcomeScreen(onToggleTheme: widget.onToggleTheme)),
    //     (route) => false,
    //   );
  }
```

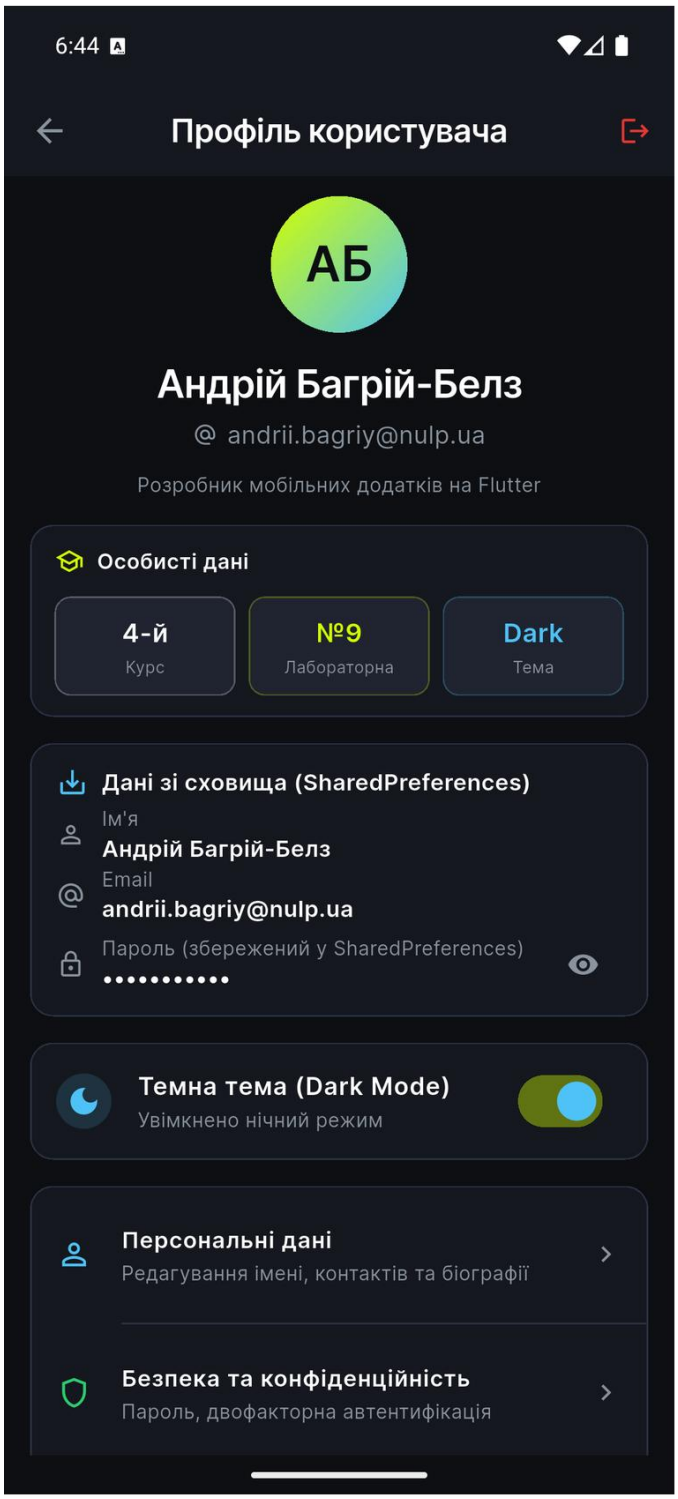
2 Результат виконання коду:



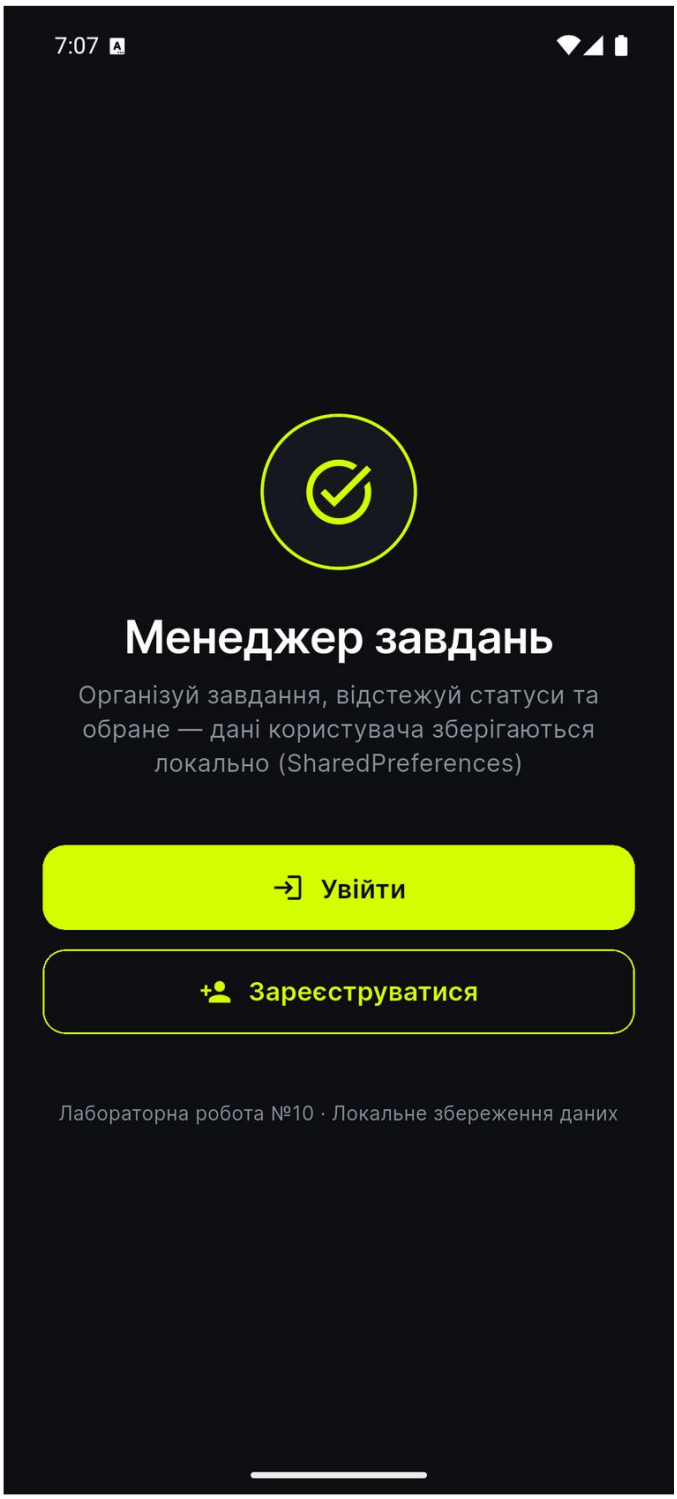
Менеджер завдань після реєстрації



Після перезапуску: користувач та обране відновлені



ProfileScreen: картка збережених даних



Після виходу: сховище очищено

1. Що таке SharedPreferences і де зберігаються дані? SharedPreferences — офіційний пакет Flutter для збереження пари «ключ–значення» у локальному сховищі платформи: на Android це XML-файл shared_prefs/FlutterSharedPreferences.xml, на iOS — NSUserDefaults. База даних і сервер для цього не потрібні.

2. Чому SharedPreferences.getInstance() викликають із await? Доступ до сховища платформи відбувається крізь platform-канал (асинхронний виклик), тому метод повертає Future. Заміну на синхронний доступ не передбачено — усі зчитування/записи в застосунку зроблені асинхронно.

3. Які типи даних може зберігати SharedPreferences? bool, int, double, String, а також їхні списки (setStringList, getIntList тощо). Саме тому список обраних збережено як List<String> через prefs.setStringList('favorite_ids', ids).
4. Чому зберігати favorites ids, а не цілі об'єкти завдань? Мінімальний обсяг даних і єдине джерело істини: повні дані завдань вже зберігає JSON-файл, а локально зберігаємо лише ідентифікатори (п. 7 ТЗ) і після запуску просто звіряємо їх зі списком (п. 8-9 ТЗ).
5. Як відновлюється сесія користувача після перезапуску? У initState вступного екрана викликається loadUser(): якщо прапорець is_registered дорівнює true, створюється об'єкт UserData і за допомогою pushReplacement застосунок одразу переходить у менеджер завдань зі збереженими іменем, email та паролем.
6. Як реалізувати додавання елемента у список обраного зі збереженням? При натисканні іконки змінюється прапорець task.isFavorite, після чого _persistFavoriteIds() збирає id усіх обраних завдань і записує їх списком через saveFavoriteIds. Той самий метод викликається при видаленні завдання та оновленні.
7. Чим SharedPreferences відрізняється від локальної бази даних (наприклад, sqflite)? SharedPreferences призначений для невеликих об'ємів налаштувань і даних користувача без запитів та зв'язків; sqflite потрібен для великих структурованих множин із вибірковим пошуком. Для задач ЛР 10 (дані профілю + список id) достатньо SharedPreferences.
8. Як видалити збережені дані (реалізувати вихід)? Окремий ключ — prefs.remove(key), а повне очищення сховища — prefs.clear(). У застосунку кнопка виходу з профілю викликає UserStorageService.clearAll(), після чого стек навігації очищується і відображається вступний екран.
9. Як тестувати код, що використовує SharedPreferences? У тестах викликають SharedPreferences.setMockInitialValues({}) — це ініціалізує в пам'яті mock-реалізацію без platform-каналу. Так у 13 автотестах застосунку перевірено збереження/завантаження даних користувача, favorites ids та сценарії входу, реєстрації й виходу.
10. Нащо прапорець is_registered? Він відзначає, що користувач уже пройшов реєстрацію, і дозволяє loadUser() швидко визначити, чи повертати об'єкт UserData, або null (коли полів у сховищі ще немає). Прапорець очищується разом з іншими ключами при виході з акаунта.

Висновок: На цій лабораторній роботі я ознайомився з локальним збереженням даних у Flutter-застосунку за допомогою пакета SharedPreferences: створив сервіс UserStorageService, який зберігає дані користувача з екрана реєстрації (ім'я, email, пароль) та список ідентифікаторів обраних елементів, реалізував автоматичне відновлення сесії після повторного запуску застосунку, відображення збережених даних на екрані профілю та відновлення списку обраного за збереженими id. Працювали всі функції: після перезапуску дані профілю не втрачалися, обрані елементи залишалися у списку, а вихід з акаунта очищав локальне сховище.

Коректність збереження перевірено 13 автоматизованими тестами та запуском на Android-емуляторі.